

ChronoQuest: Documentation & Architecture

Overall Design of the Solution

ChronoQuest is a mobile-first, web-based RPG dungeon crawler designed to demonstrate the power of intelligent agents in gaming. The solution shifts the traditional paradigm of static game logic to a dynamic, AI-orchestrated environment. The entire game loop (exploration, combat, narrative, and progression) is driven by an underlying "Antigravity AI" system that observes player behavior and adjusts the game in real-time.

Designed for a 390x844px mobile viewport, the user interface uses a sleek dark glassmorphism theme with CSS animations, ensuring high player retention and a premium feel. The core philosophy is **Transparency**—the AI DM doesn't just change stats behind the scenes; it explains its reasoning directly to the player.

Brief Overview of Architecture

The application is built using **React** and **Vite** as a Single Page Application (SPA).

- **Frontend Layer (React):** Handles state management (`gameState.js`), routing between screens (Start, Hero Select, Dungeon, Combat, Inventory), and rendering animations.
- **Agent Layer:** A suite of pure JavaScript modules acting as distinct "Agents". They intercept game events (like an attack or floor transition), process the data, and return modified parameters or narrative text.
- **Data Layer:** `LocalStorage` is utilized to persist the High Score Leaderboard across sessions, while **React state** (`useReducer/useState`) maintains the volatile state of the current dungeon run.

Mock / Real APIs Used

As a standalone client-side application designed for instant loading and zero-latency combat, ChronoQuest currently utilizes **Mock / Local APIs** powered by its internal Agent systems rather than external HTTP REST APIs.

- **Mock LLM API (NarrativeEngine):** Simulates a connection to a Large Language Model by dynamically generating contextual dialogue, boss taunts, and floor descriptions based on game state parameters.
- **Mock Analytics API (DungeonMaster):** Simulates a backend telemetry service by tracking player attack/defend ratios and win streaks in local memory to dynamically adjust game difficulty.
- **Local Storage API (Real):** The browser's native `window.localStorage` API is used to persistently store and retrieve the top 10 player scores.

Agents Developed

Three distinct, specialized Antigravity Agents were developed to orchestrate the game:

1. **The Dungeon Master (`DungeonMaster.js`):**
 - *Role:* Orchestrator & Difficulty Balancer
 - *Function:* Analyzes player win/loss streaks and playstyle (aggressive vs. defensive). It scales enemy power (0.7x - 2.0x) and selects counter-tactics to keep the game challenging but fair. It also generates the procedural dungeon layouts.
2. **The Combat Referee (`CombatReferee.js`):**
 - *Role:* Validator & Rule Enforcer
 - *Function:* Acts as the secure authority for all combat math. It calculates damage variance, enforces MP costs for special moves, handles speed-based critical hits, and strictly validates item usage and fleeing chances. All rewards are verified by the Referee.
3. **The Narrative Engine (`NarrativeEngine.js`):**
 - *Role:* Dynamic Storyteller
 - *Function:* Injects flavor and lore into the game. It provides contextual text for floor introductions, reacts to the specific enemy being fought, and drives the "AI DM Commentary" shown in the Antigravity Console.

Integrations Implemented

- **Antigravity Agentic Integration:** The core integration is the simulated linkage to the Antigravity Agent ecosystem. The UI explicitly exposes this via the "Antigravity Console", revealing the AI's internal reasoning (e.g., "🎲 Win streak detected — scaling up enemy stats").
- **Vite Build System:** Integrated for lightning-fast HMR (Hot Module Replacement) during development and optimized, minified bundling for production.
- **React State Integration:** Complex state mapping between the three autonomous agents and the React component lifecycle, ensuring that agent decisions immediately trigger UI re-renders and typewriter animations.